Bangladesh University of Engineering and Technology
Department of Computer Science and Engineering

CSE 216 — Database Sessional

Course Project — Guidelines for the 60% Project Evaluation

1. Purpose of This Document

The course project is evaluated in successive milestones. The first milestone (40%) has already been completed and assessed; it established the data layer of your application. This document states clearly (a) what you are assumed to have already delivered, and (b) exactly what is expected of you in the remaining 60% evaluation.

Read this document in full before you plan your work. Everything listed under Section 3 is mandatory unless explicitly marked optional. Anything you built in the 40% milestone that was flagged as incomplete or incorrect must be fixed before this evaluation — the 60% demonstration builds directly on top of it.

2. Already Covered in the 40% Milestone

By this point, each team is expected to already have the following in place. These items are treated as the baseline and will not be awarded fresh marks, but a regression in any of them will cost marks in the final evaluation.

2.1 Database Design

- **ERD:** A complete Entity–Relationship Diagram (ERD) for the chosen problem domain, with entities, attributes, keys and cardinalities clearly marked.
- **Relational schema:** A relational schema derived from the ERD, with sensible normalisation (at least up to 3NF unless a deliberate, justified exception was made).
- **Constraints:** Primary keys, foreign keys, NOT NULL, UNIQUE, CHECK and DEFAULT constraints applied where they are semantically required — not merely where they were convenient.
- **References:** Foreign key relationships wired correctly across tables, with appropriate ON DELETE / ON UPDATE behaviour.
- **Junction tables:** Junction (bridge) tables implemented for every many-to-many relationship, with composite or surrogate keys as appropriate.

2.2 Backend Foundation

- **Database connectivity:** A working connection from your backend application to the database server, using a connection pool or a properly managed client rather than ad-hoc connections.
- **Initial API routes:** An initial set of working API routes that read from and write to the database, demonstrating that the end-to-end path (client → backend → database → response) functions.
- **Scripts:** Schema creation scripts (DDL) and, where used, seed/sample data scripts kept in the repository.
- **Authentication routes**

3. Requirements for the 60% Evaluation

The final milestone shifts the focus from data modelling to a functioning, access-controlled application. Four areas must be covered.

3.1 Authentication (All Roles)

You must implement a complete authentication mechanism that works for every role defined in your system. Authentication answers the question “who is this user?”

- **Sign-up and login:** User registration and login must be functional for each role your project defines (for example: Admin, Manager, Staff, Customer — your role names will depend on your domain).
- **Credential storage:** Passwords must never be stored in plain text. Use a standard password hashing algorithm (bcrypt, argon2, scrypt or an equivalent) with a per-user salt. Storing an unsalted MD5/SHA-1 hash will be treated as incorrect.
- **Session management:** Maintain login state using a session or a signed token (for example a JWT stored securely, or a server-side session with an HTTP-only cookie). Be prepared to explain your choice.
- **Logout:** A working logout that genuinely invalidates the session or token — not merely a redirect on the frontend.
- **Validation and error handling:** Reject invalid credentials, empty fields, duplicate registrations and malformed input with meaningful error responses and correct HTTP status codes (400, 401, 409 as applicable).
- **Role storage:** The role of each user must be persisted in the database and resolved from there at login time — it must not be sent by the client or hard-coded.

During the demonstration you must be able to log in as every role, one after another, from a clean state.

3.2 Authorization (Role Separation Must Be Demonstrated)

Authorization answers the question “what is this authenticated user allowed to do?” This is the most heavily scrutinised requirement of the milestone. It is not enough to have roles stored in the database; you must prove that they behave differently and that they are isolated from one another.

What you must show

1. **Distinct capability per role.** Each role, once logged in, sees and can operate only the features that belong to it. Differences between roles must be visible, not theoretical.
2. **Cross-role access must be blocked.** Attempting an action that belongs to another role must fail. For example, a Customer calling an Admin-only endpoint must receive 403 Forbidden, and an unauthenticated request must receive 401 Unauthorized.
3. **Object-level ownership checks.** A user must not be able to read, modify or delete another user's data by changing an identifier in the request (for example, /orders/17 belonging to a different customer). This is checked explicitly.
4. **Server-side enforcement.** Authorization must be enforced in the backend. Hiding a button on the frontend is presentation, not security. If a protected endpoint can be reached directly with a tool such as Postman or curl, the requirement is not met.

3.3 HTTP/API Requests for at Least 20% of the Features

At least 20% of the features listed in your project proposal must be implemented through working HTTP/API endpoints.

- **REST conventions:** Use appropriate HTTP methods and resource-oriented paths, and return correct status codes (200, 201, 204, 400, 401, 403, 404, 409, 500). Search internet for relevant knowledge.
- **Input validation and SQL safety:** Validate and sanitise every input. All database access must use parameterized queries; string-concatenated SQL will be penalised as an SQL-injection defect.

3.4 Minimal Frontend

A minimal but genuinely functional frontend is required. It does not need to be visually polished; it must, however, exercise the backend end to end. Any framework or plain HTML/CSS/JavaScript is acceptable.

- **Authentication screens:** Login, registration and logout screens wired to the authentication endpoints.
- **Role-aware interface:** After login, the interface must reflect the user's role — a different landing view, menu or set of available actions per role. This is the visible counterpart of Section 3.2.
- **Feature access:** Every implemented feature should be reachable from the UI: forms for creating and updating records, lists or tables for viewing them, and controls for deletion where applicable.
- **Error feedback:** Show server-side validation messages and error responses to the user instead of failing silently or crashing.

No marks are awarded for aesthetics. Marks are awarded for a frontend that demonstrably drives the API and makes role differences visible.

4. Common Mistakes to Avoid

- Enforcing access control only on the frontend while leaving endpoints open.
- **Using ORM is strictly prohibited. You must write raw SQL queries. You can build own ORM.**
- Storing passwords in plain text, or hashing them without a salt.
- Sending the user's role from the client and trusting it on the server.
- Building SQL queries by string concatenation of user input.
- Checking that a user is logged in, but not that the record they are touching belongs to them.
- Leaving credentials, connection strings or API keys committed in the repository.
- Dividing the work such that a member cannot explain any part of the submission.

For any clarification regarding these requirements, contact the course coordinator.